

Dokumentacja Biblioteki PUTM EV CAN

Michał Błotniak

7 marca 2026

Streszczenie

Niniejszy dokument stanowi instrukcję obsługi biblioteki `PUTM_EV_CAN_LIBRARY`. Biblioteka została zaprojektowana dla systemów wbudowanych (STM32) oraz środowiska ROS2. Główne cechy to: statyczna alokacja pamięci, obsługa wielu rodzin STM32 (F4, G4, L4, H7) dzięki warstwie abstrakcji HAL, odbiór danych oparty na przerwaniach (Interrupt-Driven). Dokument zawiera również zbiór dobrych praktyk oraz najczęstszych pułapek sprzętowo-programowych.

Spis treści

1	Wymagania systemowe i zależności	2
1.1	Instalacja zależności Python	2
2	Integracja i Konfiguracja w CMake	3
2.1	Dodawanie biblioteki do projektu	3
2.2	Wybór konfiguracji (STM32 vs ROS2)	3
3	Konwencja Nomenklatury Magistral	4
4	Przygotowanie Sprzętowe (STM32CubeMX)	5
4.1	Obowiązkowa Konfiguracja FDCAN w CubeMX	5
5	Poprawna Inicjalizacja Oprogramowania	6
5.1	Krok 1: Instancje Globalne	6
5.2	Krok 2: Wywołanie Init() i Ciche Błędy	6
6	Wysyłanie (TX) i Odbieranie danych (RX)	7
6.1	Odbieranie poprzez Callbacki (Interrupt-Driven)	7
6.2	Wysyłanie Danych	7
7	Rozwiązywanie Problemów i Znane Pułapki (FAQ)	8
7.1	Pułapki Software'owe	8
7.2	Pułapki Sprzętowe (Hardware)	8

1 Wymagania systemowe i zależności

Prawidłowa kompilacja i działanie biblioteki PUTM_EV_CAN_LIBRARY wymaga zapewnienia odpowiedniego środowiska programistycznego. Poniższa tabela przedstawia minimalne wersje narzędzi oraz bibliotek wykorzystywanych w projekcie.

Narzędzie / Biblioteka	Wersja Minimalna	Uwagi
CMake	3.16	Wymagane przez konfigurację projektu.
Standard C++	C++20	Wymuszone w pliku <code>CMakeLists.txt</code> .
Standard C	C11	Standard dla kodu C.
Python	3.8+	Niezbędny do uruchomienia skryptów generujących kod z DBC.
Kompilator C++ (ARM)	$\text{GCC} \geq 10.0$	Musi obsługiwać flagi standardu C++20.
Biblioteka HAL	STM32XXxx HAL	Zależność sprzętowa dla mikrokontrolerów różnych serii.
Python: cantools	40.7.1	Generator kodu C z plików DBC.
Python: python-can	(auto)	Instalowana automatycznie jako zależność <code>cantools</code> .

Tabela 1: Wymagania systemowe projektu

1.1 Instalacja zależności Python

Skrypt generujący kod (`dbc/setup.py`) jest uruchamiany automatycznie przez CMake i wymaga zainstalowanych bibliotek w środowisku Pythona. Brak tych bibliotek spowoduje błąd konfiguracji CMake na etapie `Checking and regenerating DBC files`.

```
1 pip install cantools==40.7.1
```

Listing 1: Instalacja wymaganych bibliotek Python

2 Integracja i Konfiguracja w CMake

Biblioteka jest dystrybuowana jako moduł CMake. Aby użyć jej w projekcie, należy zmodyfikować główny plik CMakeLists.txt projektu.

2.1 Dodawanie biblioteki do projektu

BARDZO WAŻNE: Zawsze używaj słowa kluczowego PRIVATE lub PUBLIC przy linkowaniu. Dodatkowo, poprawne podlinkowanie putm_ev_can gwarantuje przekazanie definicji środowiskowych (np. -DPUTM_CAN_BACKEND_STM32). Brak poprawnego linkowania zaowocuje błędem kompilacji: `#error "Backend not defined! Check CMake configuration."`

```
1 # 1. Dodanie podkatalogu biblioteki
2 add_subdirectory(Core/Inc/PUTM_EV_CAN_LIBRARY)
3
4 # 2. Linkowanie biblioteki z twoim projektem (Zawsze używaj PRIVATE)
5 target_link_libraries(${CMAKE_PROJECT_NAME} PRIVATE
6     stm32cubemx
7     putm_ev_can
8 )
```

Listing 2: Podstawowa konfiguracja CMake

2.2 Wybór konfiguracji (STM32 vs ROS2)

System CMake automatycznie próbuje wykryć środowisko (Bare-metal STM32 vs system Linux z załadowanym ROS2). Jeśli autodetekcja zawiedzie, możesz wymusić zachowanie ręcznie.

```
1 # Dla STM32:
2 set(PUTM_CAN_BACKEND "STM32" CACHE STRING "Select backend" FORCE)
3
4 # Dla ROS2 / Linux:
5 set(PUTM_CAN_BACKEND "ROS2" CACHE STRING "Select backend" FORCE)
```

Listing 3: Wymuszanie konfiguracji w CMake

3 Konwencja Nomenklatury Magistral

W projekcie pojazdu zdefiniowano trzy fizyczne magistrale CAN. Aby zachować spójność w kodzie, **należy bezwzględnie** stosować poniższe nazewnictwo instancji klasy sterownika (np. **CanDriver** lub **AppCAN** zależnie od wersji wrappera).

Nazwa Instancji	Funkcja Magistrali	Opis
can_m	Main	Główna magistrala, większość danych, telemetria.
can_pt	Powertrain (ECU/INV)	Komunikacja wyłącznie falownik - silniki.
can_dv	Driverless	Wymiana sygnałów w trybie jazdy autonomicznej.

Tabela 2: Obowiązująca nomenklatura instancji

4 Przygotowanie Sprzętowe (STM32CubeMX)

To najważniejszy krok przy korzystaniu z nowych mikrokontrolerów z serii STM32G4 oraz STM32H7 (posiadających peryferium **FDCAN**).

ZAKAZ: Nie zostawiaj domyślnych ustawień dla FDCAN w CubeMX. FDCAN, w przeciwieństwie do starszego bxCAN (z serii F4), **wymaga ręcznej alokacji pamięci RAM dla buforów FIFO!** Jeśli tego nie zrobisz, sprzęt odrzuci każdą ramkę do wysłania bez wieszania programu (tzw. "cichy błąd").

4.1 Obowiązkowa Konfiguracja FDCAN w CubeMX

W zakładce FDCAN1/FDCAN2 → Parameter Settings należy ustawić:

- **Tx FIFO/Queue Mode:** Tx FIFO operation (Bardzo ważne!)
- **Tx FIFO/Queue Elmts Nbr:** 8 (lub inna wartość > 0. Ustawienie 0 oznacza brak pamięci do wysyłania!).
- **Rx FIFO0 Elmts Nbr:** 8 (Wymagane do odbioru z powiadomieniami).
- **Auto Retransmission:** DISABLE. Włączenie tej funkcji może doprowadzić do zapchania magistralii.

5 Poprawna Inicjalizacja Oprogramowania

Biblioteka przejmuje całkowitą kontrolę nad peryferium CAN. Inicjalizuje filtry, aktywuje powiadomienia o przerwaniach i fizycznie uruchamia magistralę.

5.1 Krok 1: Instancje Globalne

W pliku `main.cpp` (lub pliku obsługującym konkretne zadanie FreeRTOS, np. `communication_task.cpp`) należy utworzyć obiekty dla wykorzystywanych magistral bez słowa `extern`, jeśli definicja znajduje się w tym samym pliku.

```
1 // Wlasciwa definicja obiektu w pliku .cpp
2 putm_ev_can::CanDriver can_m;
```

5.2 Krok 2: Wywołanie Init() i Ciche Błędy

ZAKAZ PODWÓJNEJ INICJALIZACJI: Nie wolno wywoływać funkcji `HAL_FDCAN_Start()` oraz `HAL_FDCAN_ActivateNotification()` z poziomu pliku `main.c`! Biblioteka wewnątrz metody `can_m.Init(&hfdcan1)` sama to robi. Jeśli wywołasz Start sprzętowy przed biblioteką, funkcja `Init()` zwróci błąd (status *BUSY*), flaga wewnętrzna biblioteki pozostanie w stanie *uninitialized*, a funkcja `Send()` nigdy nie wyśle ramki.

Dobra Praktyka (Poprawny kod):

```
1 /* USER CODE BEGIN 2 */
2 // Tylko konfiguracja filtrow (jesli potrzebna)
3 FDCAN_FilterTypeDef filter_config;
4 // ... ustawienia filtra ...
5 HAL_FDCAN_ConfigFilter(&hfdcan1, &filter_config);
6
7 // ZADNYCH HAL_FDCAN_Start() TUTAJ! Zostaw to bibliotece.
8 /* USER CODE END 2 */
9
10 // ... pozniej, w tasku lub przed petla glowna ...
11 void Communication_Task(void* argument) {
12     // 1. To wystarczy do uruchomienia komunikacji
13     if (!can_m.Init(&hfdcan1)) {
14         // Bład inicjalizacji (np. zablokowany sprzet)
15         Error_Handler();
16     }
17     // ...
18 }
```

6 Wysyłanie (TX) i Odbieranie danych (RX)

6.1 Odbieranie poprzez Callbacki (Interrupt-Driven)

Odbiór w nowej wersji biblioteki działa asynchronicznie. **UWAGA:** Kod wewnątrz callbacku wykonuje się w **KONTEKŚCIE PRZERWANIA SPRZĘTOWEGO (ISR)**.

ZAKAZY wewnątrz Callbacku:

- Nie używaj funkcji blokujących (np. `osDelay`, `HAL_Delay`).
- Nie wykonuj ciężkich obliczeń matematycznych.
- Nie wywołuj funkcji typu `printf`.

Dobra Praktyka: Zmieniaj stan zmiennych oznaczonych jako `volatile`, odświeżaj watchdoga lub używaj `osMutexAcquire(..., 0)` (czas oczekiwania 0 dla przerwania!).

```
1 // Rejestracja musi odbyć się tylko raz!
2 can_m.RegisterCallback<PUTM_CAN_M_front_data_t>(
3     PUTM_CAN_M_FRONT_DATA_FRAME_ID,
4     [](const PUTM_CAN_M_front_data_t& front_data) {
5
6         HAL_IWDG_Refresh(&hiwdg); // Szybka, dozwolona operacja w ISR
7
8         // Blokada mutexa z czasem = 0 (aby nie zablokować przerwania)
9         if (osMutexAcquire(dataMutexHandle, 0) == osOK) {
10             data.brake_light = front_data.is_braking;
11             osMutexRelease(dataMutexHandle);
12         }
13     }
14 );
```

Listing 4: Rejestracja callbacku i bezpieczne operacje

6.2 Wysyłanie Danych

Metoda `Send` sprawdza typ i bezpiecznie umieszcza ramkę w kolejce (dlatego konfiguracja FIFO w CubeMX była tak kluczowa).

```
1 // Przykład w p tli for(;;)
2 PUTM_CAN_M_rearbox_temperature_t temp_msg = {
3     .mono_temperature = (uint8_t)temperature.mono,
4     .coolant_temperature_in = (uint8_t)temperature.coolant_in,
5     .coolant_temperature_out = (uint8_t)temperature.coolant_out,
6     .oil_temperature_l = (uint8_t)temperature.oil_l,
7     .oil_temperature_r = (uint8_t)temperature.oil_r
8 };
9
10 // Dobrą praktyką jest weryfikacja czy ramka została dodana do kolejki
11 if (!can_m.Send(PUTM_CAN_M_REARBOX_TEMPERATURE_FRAME_ID, temp_msg)) {
12     // np. zapal czerwony diodę na uk adzie TCA.
13 }
```

7 Rozwiązywanie Problemów i Znane Pułapki (FAQ)

Poniżej zestawiono najczęściej spotykane problemy, objawy oraz rozwiązania sprzętowo-programowe.

7.1 Pułapki Software'owe

- **Objaw:** Kod się wykonuje, ale ramki nie wychodzą na zewnątrz (`Send` zwraca błąd).
 - **Przyczyna 1:** W CubeMX nie zdefiniowano rozmiaru kolejek FDCAN (`Tx FIFO/Queue Elmts Nbr` równe 0).
 - **Przyczyna 2:** Przedwczesne uruchomienie sprzętu przez `HAL_FDCAN_Start()` przed zainicjowaniem biblioteki.
- **Objaw:** Błąd w CMake "Backend not defined".
 - **Przyczyna:** Brak podlinkowania biblioteki do targetu przez `target_link_libraries(... PRIVATE putm_ev_can)`.

7.2 Pułapki Sprzętowe (Hardware)

- **Objaw:** Brak sygnału na magistrali widocznego na oscyloskopie, lub jest on całkowicie niezrozumiały/zniekształcony, a debugowanie oprogramowania nie przynosi efektu.
 - **Przyczyna:** Magistrala CAN jest niezamknięta. **Brak rezystorów terminujących 120 Ω (Ohm) pomiędzy pinami CAN_H oraz CAN_L.** Sieć CAN działa w oparciu o różnicę napięć i wymaga fizycznego "ściągnięcia" linii do stanu recesywnego. Podczas testów "na biurku" upewnij się, że Twój sprzętowy analizator CAN ma wbudowany rezystor, lub dodaj go "na krótko" we wtyczce.
- **Objaw:** Transiwer CAN się nie odzywa.
 - **Przyczyna:** Pin `STBY` transiweru CAN nie jest ściągnięty do masy (GND). Zweryfikuj, czy odpowiednie rezystory konfiguracyjne zostały wlutowane na PCB.